



Exponent CLMM

Security Assessment

December 22nd, 2025 — Prepared by OtterSec

Ajay Shankar Kunapareddy

d1r3wolf@osec.io

Xiang Yin

soreatu@osec.io

Robert Chen

r@osec.io

Table of Contents

Executive Summary	3
Overview	3
Key Findings	3
Scope	4
Findings	5
Vulnerabilities	6
OS-ELM-ADV-00 Inconsistency in SY-Side Minting Logic	10
OS-ELM-ADV-01 Inconsistent Farm Budget Calculation	11
OS-ELM-ADV-02 Improper Prefix Sum Adjustment	13
OS-ELM-ADV-03 Erroneous Current Tick Reassignment Condition	14
OS-ELM-ADV-04 Improper Utilization of SY Constraint in SellYt	15
OS-ELM-ADV-05 Frontrunning Risk in Token Account Creation	16
OS-ELM-ADV-06 Failure to Deduct Fees from Market Financials	17
OS-ELM-ADV-07 Unsafe Admin Emission Removal	18
OS-ELM-ADV-08 Lack of Inclusion of Boundary Ticks	19
OS-ELM-ADV-09 Incorrect Post-Split Indexing	21
OS-ELM-ADV-10 Unnecessary Emission Tracker Updates	22
OS-ELM-ADV-11 Improper LpPosition Deserialization Offset	23
OS-ELM-ADV-12 Faulty Sentinel Key Handling	24
OS-ELM-ADV-13 Farm Reward Distribution Calculation Discrepancy	25
OS-ELM-ADV-14 Premature Placement of LP Supply Check	26
OS-ELM-ADV-15 Duplicate Account Initialization	27

OS-ELM-ADV-16 Incorrect Invocation of Timestamp Function	28
OS-ELM-ADV-17 Lack of Tracker Limit Checks	29
OS-ELM-ADV-18 Improper Size Allocations	30
OS-ELM-ADV-19 Balance Limit bypass due to Round Down Division	33
OS-ELM-ADV-20 Inflated Emission Index Initialization	34
OS-ELM-ADV-21 Missing Final Fee Index Recording	35
OS-ELM-ADV-22 Unrestricted Market Initialization	36
General Findings	37
OS-ELM-SUG-00 Missing Functionality to Update Admin and Treasury Fields	38
OS-ELM-SUG-01 Code Refactoring	39
OS-ELM-SUG-02 Validation for CPI Accounts	42
OS-ELM-SUG-03 Discrepancy in SY Need Calculation	44
 Appendices	
Vulnerability Rating Scale	45
Procedure	46

01 — Executive Summary

Overview

Exponent Finance Labs engaged OtterSec to assess the `cLmm` program. This assessment was conducted between October 13th and November 26th, 2025. For more information on our auditing methodology, refer to [Appendix B](#).

Key Findings

We produced 27 findings throughout this audit engagement.

In particular, we identified a vulnerability where the SY-side non-crossing deposit path bypasses LP share minting and pre-state validation, allowing SY principal to be credited without enforcing liquidity caps or updating emissions ([OS-ELM-ADV-00](#)), and another issue where the token farm amount, which includes unclaimed staged rewards, is incorrectly utilized to adjust farm funds, potentially allowing premature withdrawal of tokens needed for payment of these staged emissions ([OS-ELM-ADV-01](#)). Also, there is a risk of frontrunning the token account creation function, where an attacker may pre-fund the target account, resulting in failing the account creation ([OS-ELM-ADV-05](#)).

Additionally, the emission trackers are incorrectly updated during tick crossings, even though emissions depend on deposited SY, and not swap activity ([OS-ELM-ADV-10](#)), and when depositing liquidity, the LP supply check is executed before liquidity is added, allowing deposits to exceed the maximum LP supply ([OS-ELM-ADV-14](#)). Moreover, the market initialization function fails to check if the SY personal account already exists, resulting in a revert if the lp position has already been created ([OS-ELM-ADV-15](#)).

Furthermore, several structures and functions contain incorrect size calculations and missing field allocations, resulting in potential serialization or memory layout mismatches ([OS-ELM-ADV-18](#)), and the unrestricted access to the market initialization instruction allows anyone to create markets, enabling attackers to exhaust the limit of 256 seed IDs and block legitimate market creation ([OS-ELM-ADV-22](#)),

We also made recommendations to add functionality for updating fee treasury and admin accounts, and to implement a two-step process for admin change to prevent irreversible ownership transfers ([OS-ELM-SUG-00](#)). Moreover, we suggested refactoring the codebase for better functionality and addressing inconsistencies ([OS-ELM-SUG-01](#)). We further advised implementing appropriate account validations for CpiAccounts and CpiCoreAccounts ([OS-ELM-SUG-02](#)).

02 — Scope

The source code was delivered to us in a Git repository at <https://github.com/exponent-finance/exponent-core>. This audit was performed against `889bca0`.

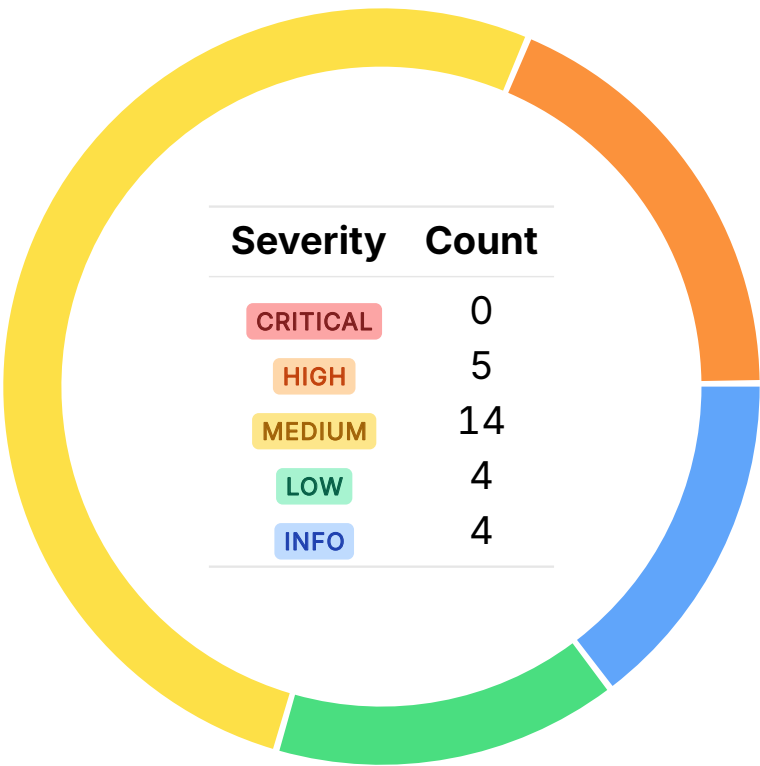
A brief description of the program is as follows:

Name	Description
clmm	A concentrated liquidity AMM on Solana, enabling users to trade, provide liquidity, and earn emissions across Principal, Yield, and Synthetic Yield tokens within configurable on-chain markets.

03 — Findings

Overall, we reported 27 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in [Appendix A](#).

ID	Severity	Status	Description
OS-ELM-ADV-00	HIGH	RESOLVED ✓	The <code>SY</code> -side non-crossing deposit path bypasses LP share minting and pre-state validation, allowing <code>SY</code> principal to be credited without enforcing liquidity caps or updating emissions.
OS-ELM-ADV-01	HIGH	RESOLVED ✓	<code>modify_farm</code> incorrectly utilizes <code>token_farm.amount</code> , which includes unclaimed staged rewards, to adjust farm funds, potentially allowing premature withdrawal of tokens needed for payment of these staged emissions.
OS-ELM-ADV-02	HIGH	RESOLVED ✓	In <code>accrue_principal_for_deposit</code> , when <code>l_actual < l_target</code> tick liquidity is correctly reduced but <code>current_prefix_sum</code> is not reduced, rendering active-liquidity accounting inconsistent.
OS-ELM-ADV-03	HIGH	RESOLVED ✓	The current condition in <code>upsert_boundary_tick</code> updates <code>current_tick</code> too broadly, allowing any lower-priced, lower-key tick to replace it even if it is not the correct boundary.
OS-ELM-ADV-04	HIGH	RESOLVED ✓	The <code>sell_yt</code> instruction misuses exact-input swap semantics for a swap that requires an exact-output configuration, passing <code>yt_in</code> as the input amount instead of the required <code>PT</code> output.

OS-ELM-ADV-05	MEDIUM	RESOLVED ✓	<code>create_token_account</code> is vulnerable to frontrunning, enabling an attacker to pre-fund the target account, resulting in <code>create_account</code> failing.
OS-ELM-ADV-06	MEDIUM	RESOLVED ✓	Protocol fees and accrued liquidity provider fees paid out in <code>trade_pt</code> and <code>remove_liquidity</code> are not deducted from <code>market.financials</code> , resulting in financials that overstate the protocol's actual token balances, creating accounting inconsistencies.
OS-ELM-ADV-07	MEDIUM	RESOLVED ✓	It is possible for admins to remove market emissions which undermines liquidity provider expectations.
OS-ELM-ADV-08	MEDIUM	RESOLVED ✓	<code>get_amount_ticks_in_range</code> excludes both boundary ticks, counting only interior ticks and producing incorrect tick-range sizes.
OS-ELM-ADV-09	MEDIUM	RESOLVED ✓	The split logic does not update <code>right_tick_idx</code> after a tick is divided.
OS-ELM-ADV-10	MEDIUM	RESOLVED ✓	Currently, emission trackers are incorrectly updated during tick crossings, even though emissions depend on deposited <code>SY</code> , and not swap activity.
OS-ELM-ADV-11	MEDIUM	RESOLVED ✓	<code>wrapper_deposit_liquidity_classic</code> and <code>wrapper_withdraw_liquidity</code> skip the Anchor discriminator by deserializing from <code>lp_position_data[8..]</code> .
OS-ELM-ADV-12	MEDIUM	RESOLVED ✓	<code>get_amount_ticks_in_range</code> incorrectly utilizes <code>SENTINEL</code> as a tick key when no valid start key exists, resulting in a real tick with key zero to be misinterpreted as inside the user's range.

OS-ELM-ADV-13	MEDIUM	RESOLVED ✓	<code>increase_share_indexes</code> utilizes total market liquidity instead of tick-level LP contributions, resulting in inaccurate distribution of farm rewards.
OS-ELM-ADV-14	MEDIUM	RESOLVED ✓	In the <code>deposit_liquidity</code> instruction, the LP supply check runs before liquidity is added, allowing deposits to exceed <code>max_lp_supply</code> .
OS-ELM-ADV-15	MEDIUM	RESOLVED ✓	The market initialization function fails to check if the <code>SY</code> position account already exists, resulting in a revert if the <code>lp_position</code> has already been created.
OS-ELM-ADV-16	MEDIUM	RESOLVED ✓	<code>withdraw_liquidity</code> and <code>trade_pt</code> instructions mistakenly cast the <code>now</code> function pointer to <code>u32</code> instead of calling <code>now</code> , resulting in incorrect timestamp utilization.
OS-ELM-ADV-17	MEDIUM	RESOLVED ✓	<code>add_farm</code> and <code>add_market_emission</code> do not enforce safe vector lengths.
OS-ELM-ADV-18	MEDIUM	RESOLVED ✓	Several structures and functions contain incorrect size calculations and missing field allocations, resulting in potential serialization or memory layout mismatches.
OS-ELM-ADV-19	LOW	RESOLVED ✓	In <code>LiquidityNetBalanceLimits</code> , the current computation of <code>percentage_increase</code> and <code>percentage_decrease</code> results in truncating the result, allowing a net balance change to slightly exceed the configured limits.
OS-ELM-ADV-20	LOW	RESOLVED ✓	<code>MarketEmissions::add_emission</code> incorrectly sets <code>lp_share_index</code> to one instead of zero, resulting in new LPs receiving inflated initial rewards.

OS-ELM-ADV-21	LOW	RESOLVED ✓	The swap event emits incomplete tick-modification data because the final global fee-growth indices are never written into <code>ModifiedTicks</code>
OS-ELM-ADV-22	LOW	RESOLVED ✓	The unrestricted access to <code>initialize_market</code> instruction allows anyone to create markets, enabling attackers to exhaust the limit of 256 <code>seed_ids</code> and block legitimate market creation.

Inconsistency in SY-Side Minting Logic HIGH

OS-ELM-ADV-00

Description

In `add_liquidity::accrue_principal_for_deposit`, the `SY`-side non-crossing interval path bypasses `mint_shares_for_tick_prestate_calc_used` and directly applies `add_principal_sy`, resulting in `SY` principal to be counted without minting corresponding LP shares or validating how much `SY` the tick can actually consume. This skips pre-state checks, emission index updates, and liquidity-cap enforcement that the `PT` branch correctly applies. As a result, tick accounting becomes inconsistent.

```
>_ src/state/market_three/helpers/add_liquidity.rs
```

RUST

```
pub fn accrue_principal_for_deposit([...]) -> Result<(u64, u64)> {
    [...]
    if sy_slice_start < sy_slice_end {
        [...]
        if principal_sy_delta > 0 {
            [...]
            add_principal_sy(
                segment.ticks,
                segment.left_index,
                principal_sy_delta,
                market_emission_indices,
            )?;
            total_sy_spend =
                total_sy_spend.checked_add(principal_sy_delta).unwrap();
        }
        [...]
    }
    Ok((total_sy_spend, total_pt_spend))
}
```

Remediation

Invoke `mint_shares_for_tick_prestate_calc_used` within the `SY` branch when the interval is not crossing.

Patch

Resolved in [PR#3084](#).

Inconsistent Farm Budget Calculation HIGH

OS-ELM-ADV-01

Description

In the `modify_farm` instruction, the program utilizes `token_farm.amount` to determine whether to add or remove tokens when updating farm emissions. However, this value includes unclaimed staged rewards, rendering it an unreliable indicator of the actual remaining emission budget. As a result, comparing `new_tokens_needed` with `token_farm.amount` may mistakenly identify a surplus and withdraw tokens that are still required to pay user rewards, resulting in underfunding. A similar issue exists in `exponent_core` as well.

```
>_ src/instructions/market_three/admin/modify_farm.rs
```

RUST

```
#[access_control(ctx.accounts.validate())]
pub fn handler([...]) -> Result<()> {
    [...]
    let new_tokens_needed =
        (new_expiration_timestamp as i64 - current_timestamp as i64) * new_rate as i64;

    // If the token_farm has less than the required amount, transfer the difference
    // Otherwise, transfer surplus to the token_source
    if new_tokens_needed > ctx.accounts.token_farm.amount as i64 {
        let required_amount = new_tokens_needed - ctx.accounts.token_farm.amount as i64;
        #[allow(deprecated)]
        transfer(ctx.accounts.transfer_ctx(), required_amount as u64)?;
    } else {
        let surplus_amount = ctx.accounts.token_farm.amount as i64 - new_tokens_needed;
        #[allow(deprecated)]
        transfer(
            ctx.accounts
                .transfer_from_market_ctx()
                .with_signer(&[&ctx.accounts.market.signer_seeds()]),
            surplus_amount as u64,
        )?;
    }

    Ok(())
}
```

Remediation

Calculate the remaining emission budget (`left_tokens`) as `(previous_expiration_timestamp - current_timestamp) * previous_token_rate` and compare it against `new_tokens_needed`.

Patch

Resolved in [PR#3046](#).

Improper Prefix Sum Adjustment HIGH

OS-ELM-ADV-02

Description

`add_liquidity::accrue_principal_for_deposit` adjusts tick-level liquidity when `l_actual < l_target`, but does not apply the same correction to `ticks.current_prefix_sum`. Because the prefix sum represents the currently active liquidity across ticks, and the liquidity minted for the current active interval is not exactly `liquidity_needs.liquidity_target`, it must remain synchronised with all liquidity net changes. Thus, when the code subtracts `delta_l` from the lower and upper boundary ticks, the active liquidity actually decreases, however, since `current_prefix_sum` is not reduced by the same `delta_l`, the system retains an inflated view of active liquidity, resulting in an inconsistent state.

```
>_ src/state/market_three/helpers/add_liquidity.rs
```

RUST

```
pub fn accrue_principal_for_deposit([...]) -> Result<(u64, u64)> {
    [...]
    if is_crossing {
        [...]
        // 4) if l_actual < L_target - we should reduce L in tick
        if l_actual < l_target {
            let delta_l: i64 = (l_target - l_actual).floor() as i64; // positive
            segment.ticks.tick_sub_liquidity_net_gross(
                pt_left_index as u32,
                delta_l,
                true,
            )?;
            segment.ticks.tick_sub_liquidity_net_gross(
                segment.right_index,
                delta_l,
                false,
            )?;
        }
        return Ok(());
    }
    [...]
}
```

Remediation

Reduce `ticks.current_prefix_sum` by `delta_l` when `l_actual < l_target`.

Patch

Resolved in [PR#3085](#).

Erroneous Current Tick Reassignment Condition HIGH

OS-ELM-ADV-03

Description

In `ticks::upsert_boundary_tick`, the update condition for `current_tick` is overly permissive. It overwrites `current_tick` for any newly inserted tick whose `key` and `spot_price` are both smaller than the current one, even if that tick is far away from the true price band. This allows `current_tick` to drift arbitrarily left, pointing to ticks that do not correspond to the current price.

```
>_ src/state/ticks.rs
```

RUST

```
pub fn upsert_boundary_tick([...]) -> Result<u32> {
    let optional_node_index = self.ticks_tree.get_addr(&key);
    let node_idx = if optional_node_index != SENTINEL {
        optional_node_index
    } else {
        [...]
        // Insert and recompute upwards in one go (your function)
        let node_idx = self.ticks_tree.insert(key, *tick).unwrap();
        if self.current_tick == SENTINEL
            || (self.ticks_tree.get_node(self.current_tick).key > key
                && spot_price < self.current_spot_price)
        {
            self.current_tick = node_idx;
        }
        [...]
    };
    Ok(node_idx)
}
```

Remediation

Refactor the `current_tick` assignment logic.

Patch

Resolved in [PR#3086](#).

Improper Utilization of SY Constraint in SellYt HIGH

OS-ELM-ADV-04

Description

In the `sell_yt` instruction, the protocol needs to acquire exactly `yt_in` `PT` tokens in order to repay the borrowed position. This implies an exact-output swap in the `SyToPt` direction, where `PT` is the desired output amount. However, the CPI call to `do_cpi_buy_pt` passes `yt_in` as `amount_in` and `sy_recv` as the output constraint, which corresponds to an exact-input swap.

```
>_ src/instructions/market_three/sell_yt.rs
```

RUST

```
#[access_control(ctx.accounts.validate())]
pub fn handler<'i>(<
  ctx: Context<'_, '_, '_, 'i, SellYt<'i>>,
  yt_in: u64,
  min_sy_out: u64,
  price_spot_limit: Option<f64>,
) -> Result<SellYtEvent> {
  [...]
  // perform the purchase of PT with the SY
  let sy_spent = ctx
    .accounts
    .do_cpi_buy_pt(
      ctx.remaining_accounts,
      yt_in.try_into().expect("overflow converting yt_in to i64"),
      sy_recv,
      price_spot_limit,
    )
    .expect("Trade PT failed");
  [...]
}
```

Remediation

Implement exact out swap in `SytoPT` direction.

Patch

Resolved in [PR#3105](#).

Frontrunning Risk in Token Account Creation

MEDIUM

OS-ELM-ADV-05

Description

`create_token_account` is vulnerable to front-running, enabling an attacker to pre-fund the target `token_account` address before the original instruction executes, resulting in `system_program::create_account` failing since it requires the account to have zero lamports. This effectively creates a denial-of-service scenario by preventing account initialization.

```
>_ solana/libraries/token_util/src/lib.rs
```

RUST

```
pub fn create_token_account<'a>(
    authority: &AccountInfo<'a>,
    payer: &AccountInfo<'a>,
    token_account: &AccountInfo<'a>,
    mint_account: &AccountInfo<'a>,
    system_program: &AccountInfo<'a>,
    token_program: &AccountInfo<'a>,
    signer_seeds: &[&[u8]],
) -> Result<()> {
    [...]
    let cpi_context = CpiContext::new(system_program.to_account_info(), cpi_accounts);
    anchor_lang::system_program::create_account(
        cpi_context.with_signer(signer_seeds),
        lamports,
        space as u64,
        token_program.key,
    )?;
    [...]
}
```

Remediation

Empty the account's lamports before creation, or transfer the necessary rent, allocate space, and assign ownership. This ensures the account may be safely initialized even if it is already funded.

Patch

Resolved in [PR#2955](#), [PR#2970](#).

Failure to Deduct Fees from Market Financials MEDIUM

OS-ELM-ADV-06

Description

In `trade_pt`, during a `PT` trade, the code calculates the protocol fee (`trade_result.protocol_fee_charged_out_token`) based on the swap. However, currently the protocol fee is sent to the treasury but not deducted from `market.financials`, creating accounting inconsistencies by inflating available liquidity, which skews liquidity calculations.

Similarly, the `remove_liquidity` instruction deducts principal `PT` and `SY` tokens from `market.financials` but fails to deduct the fee amounts (`fee_sy_out` and `fee_pt_out`) claimed by the liquidity provider. Omitting these deductions results in the market's internal balances to be overstated, creating a mismatch between the market's accounted token balances and the actual tokens held.

Remediation

Ensure to deduct `trade_result.protocol_fee_charged_out_token` from `market.financials` within `trade_pt`. Also, deduct `fee_sy_out` and `fee_pt_out` from `market.financials` in `remove_liquidity`

Patch

Resolved in [PR#3080](#) and [PR#3081](#).

Unsafe Admin Emission Removal

MEDIUM

OS-ELM-ADV-07

Description

`modify_market_setting` enables the admin to execute `RemoveMarketEmission`, allowing the admin to remove emissions, which introduces a high-impact governance and safety risk because it enables unilateral and irreversible modification of the market's incentive structure at runtime.

```
>_ src/instructions/market_three/admin/modify_market_setting.rs
```

RUST

```
#[access_control(ctx.accounts.validate())]
pub fn handler(ctx: Context<ModifyMarketSetting>, action: MarketAdminAction) -> Result<()> {
    let market = &mut ctx.accounts.market;

    match action {
        [...]
        MarketAdminAction::RemoveMarketEmission(emission_index) => {
            market.emissions.trackers.remove(emission_index as usize);
        }
        [...]
    }
    Ok(())
}
```

Remediation

Remove `RemoveMarketEmission` to ensure admins cannot remove emissions.

Patch

Resolved in [PR#3009](#).

Lack of Inclusion of Boundary Ticks

MEDIUM

OS-ELM-ADV-08

Description

`ticks::get_amount_ticks_in_range` under-counts ticks because it excludes the lower boundary tick when `left_idx_optional == SENTINEL`, incrementing the counter when searching for the first tick strictly greater than `lower_key`. It also fails to increment the counter when encountering a tick equal to `upper_key`, resulting in the upper boundary tick to be omitted as well. Consequently, only the interior ticks are counted rather than an inclusive range, resulting in incorrect interval sizing.

>_ src/state/ticks.rs

RUST

```
pub fn get_amount_ticks_in_range(&mut self, lower_key: u32, upper_key: u32) -> Result<usize> {
    let left_idx_optional = self.ticks_tree.get_addr(&lower_key);
    let mut counter: usize = 0;
    let mut counter_index = if left_idx_optional != SENTINEL {
        left_idx_optional
    } else {
        counter += 1;
        let start_key = if let Some(start_key_from_tree) =
            self.ticks_tree.iter_mut().find(|tick| *tick.0 > lower_key)
        {
            *start_key_from_tree.0
        }[...];
    };
    // Totally new interval
    if counter_index == SENTINEL {counter = 2; return Ok(counter); } else {
        while let Some(successor) = self.successor_idx(counter_index) {
            [...]
            else if self.ticks_tree.get_node(successor).key == upper_key {
                return Ok(counter);
            } else {
                counter += 1; // no upper tick exist
                return Ok(counter);
            }
        }
    }
    [...]
}
```

Furthermore, since `get_amount_ticks_in_range` only reads tick data, utilizing `iter_mut` is unnecessary and over-restrictive. Switch to `iter` to enforce a read-only traversal.

Remediation

Ensure the starting and ending boundary ticks are included in the count.

Patch

Resolved in [PR#3082](#) and [PR#3093](#).

Incorrect Post-Split Indexing MEDIUM

OS-ELM-ADV-09

Description

The projection logic in `project_anchor_shares_to_current_ticks` splits a principal share across new tick boundaries but fails to update the original share's `right_tick_idx` to the new `splited_index`, leaving it mapped to an outdated range. This results in liquidity removal operations to burn shares and distribute `PT` / `SY` based on incorrect interval boundaries.

Remediation

Ensure the split logic updates `principal_share`'s `right_tick_idx` to `splited_index` after a tick is divided.

Patch

Resolved in [PR#3083](#).

Unnecessary Emission Tracker Updates

MEDIUM

OS-ELM-ADV-10

Description

`cross_tick_update_outside_u128` should only update fee-growth-outside values, but it incorrectly updates emission trackers as well. Emissions are distributed strictly based on the amount of `SY` deposited into a position, not on swap-driven tick crossings. Updating emission trackers here distorts reward accounting.

Remediation

Refrain from updating `self.emissions.trackers` in `cross_tick_update_outside_u128`.

Patch

Resolved in [PR#3087](#)

Improper LpPosition Deserialization Offset

MEDIUM

OS-ELM-ADV-11

Description

Both `wrapper_deposit_liquidity_classic` and `wrapper_withdraw_liquidity` deserialize the LP position via `lp_position_data[8..]`, which strips off the 8-byte Anchor discriminator. This bypasses Anchor's built-in discriminator check and feeds `try_deserialize` raw data without verifying that the account is actually an `LpPosition`.

```
>_ src/instructions/wrappers/wrapper_withdraw_liquidity.rs
```

RUST

```
pub fn handler<'i>(<br>  ctx: Context<'_, '_, '_, 'i, WrapperWithdrawLiquidity<'i>>,<br>  amount_lp: u64,<br>  sy_constraint: u64,<br>  redeem_sy_accounts_length: u8,<br>) -> Result<()> {<br>  [...]<br>  let lp_position_data = ctx.accounts.lp_position.try_borrow_data()?;<br>  let lp_position = crate::LpPosition::try_deserialize(&mut &lp_position_data[8..])?;<br>  [...]<br>}
```

Remediation

Utilize `lp_position_data[..]` when invoking `LpPosition::try_deserialize` to ensure proper discriminator validation.

Patch

Resolved in [PR#3140](#).

Faulty Sentinel Key Handling

MEDIUM

OS-ELM-ADV-12

Description

When `ticks::get_amount_ticks_in_range` fails to find a `start_key ≥ lower_key`, it sets `start_key = SENTINEL` and then calls `self.ticks_tree.get_addr(&start_key)`. `SENTINEL` is an index sentinel (a reserved node index), not a tick key, so passing it to functions that expect a key may select an unintended key (for example key 0, which is incorrect). Thus, `get_amount_ticks_in_range` utilizes `SENTINEL` as both an invalid index and, incorrectly, as a placeholder tick key.

```
>_ src/state/ticks.rs RUST

pub fn get_amount_ticks_in_range(&mut self, lower_key: u32, upper_key: u32) -> Result<usize> {
    [...]
    let mut counter_index = if left_idx_optional != SENTINEL {
        left_idx_optional
    } else {
        counter += 1;
        let start_key = if let Some(start_key_from_tree) =
            self.ticks_tree.iter_mut().find(|tick| *tick.0 > lower_key)
        {
            *start_key_from_tree.0
        } else {
            SENTINEL
        };
        self.ticks_tree.get_addr(&start_key)
    };
    [...]
}
```

Remediation

Refactor the logic in `get_amount_ticks_in_range` to ensure proper `start_key` retrieval.

Patch

Resolved in [PR#3376](#).

Farm Reward Distribution Calculation Discrepancy

MEDIUM

OS-ELM-ADV-13

Description

In both `add_farm` (shown below) and `modify_farm`, `increase_share_indexes` is called with `market.financials.liquidity_balance`, which represents total market liquidity. However, emissions should be distributed based on tick-level LP contributions, represented by `ticks.current_prefix_sum`. Utilizing the market-level liquidity may miscalculate each LP's share of rewards.

```
>_ src/instructions/market_three/admin/add_farm.rs
```

RUST

```
#[access_control(ctx.accounts.validate())]
pub fn handler(ctx: Context<AddFarm>, token_rate: u64, until_timestamp: u32) -> Result<()> {
    let farm_token_mint = &ctx.accounts.mint_new;
    let current_unix_timestamp = Clock::get()?.unix_timestamp as u32;

    let current_lp_escrow_amount = ctx.accounts.market.financials.liquidity_balance;
    ctx.accounts
        .market
        .lp_farm
        .increase_share_indexes(current_unix_timestamp, current_lp_escrow_amount);

    [...]
}
```

Remediation

Utilize `ticks.current_prefix_sum` as the second parameter when invoking `increase_share_indexes` in `add_farm` and `modify_farm`.

Patch

Resolved in [PR#3090](#).

Premature Placement of LP Supply Check MEDIUM

OS-ELM-ADV-14

Description

The `market::check_supply_lp` validation in `deposit_liquidity::validate` is performed at the beginning of the function, before computing the new liquidity. Consequently, it only checks the current LP supply and not the final amount after the deposit. This allows a user's deposit to exceed the `max_lp_supply` limit, breaking the market's liquidity cap invariant.

```
>_ src/instructions/market_three/deposit_liquidity.rs
```

RUST

```
fn validate(&self) -> Result<()> {  
    if self.market.check_supply_lp() == false {  
        return Err(ExponentCLMMError::LpSupplyMaximumExceeded.into());  
    }  
    require!(  
        self.market.check_status_flags(STATUS_CAN_DEPOSIT_LIQUIDITY),  
        ExponentCLMMError::DepositingLiquidityDisabled  
    );  
    Ok(())  
}
```

Remediation

Place the check after liquidity addition to ensure the total supply remains within bounds.

Patch

Resolved in [PR#3005](#).

Duplicate Account Initialization MEDIUM

OS-ELM-ADV-15

Description

During market initialization in `market_three_init`, the current code always calls `cpi_init_sy_personal_account` to create an `SY` position account without checking if it already exists. In case the `lp_position` has already been created, the CPI call to `cpi_init_sy_personal_account` will fail and result in reverting the entire transaction.

```
>_ src/instructions/market_three/admin/market_three_init.rs
```

RUST

```
pub fn handler<'info>([...]) -> Result<MarketThreeInitEvent> {  
    [...]  
    // Create an account for the Market robot with the SY Program  
    cpi_init_sy_personal_account(ctx.accounts.sy_program.key(), ctx.remaining_accounts)?;  
    [...]  
}
```

Remediation

Check if the position account already exists during market initialization, and if it does, skip invoking `cpi_init_sy_personal_account`.

Patch

Resolved in [PR#3453](#).

Incorrect Invocation of Timestamp Function

MEDIUM

OS-ELM-ADV-16

Description

In both `withdraw_liquidity` (shown below) and `trade_pt` instructions, the code incorrectly utilizes `now as u32` expression, which casts the function pointer of `now` to `u32` instead of calling `now`. This results in passing a memory address rather than the actual timestamp.

```
>_ src/instructions/market_three/withdraw_liquidity.rs
```

RUST

```
#[access_control(ctx.accounts.validate(liquidity_to_remove))]  
pub fn handler<'info>([...]) -> Result<WithdrawLiquidityEvent> {  
    [...]  
    let current_active_liquidity = ticks.current_prefix_sum;  
    let sy_balance = ctx.accounts.market.financials.sy_balance;  
    ctx.accounts  
        .market  
        .update_emissions_from_position_state(&position_state, sy_balance);  
    ctx.accounts  
        .market  
        .lp_farm  
        .increase_share_indexes(now as u32, current_active_liquidity);  
    ctx.accounts.market.update_lp_position_shares(ticks,&mut ctx.accounts.lp_position)?;  
    [...]  
}
```

Remediation

Replace it with `now()` to correctly pass the current time.

Patch

Resolved in [PR#3004](#).

Lack of Tracker Limit Checks MEDIUM

OS-ELM-ADV-17

Description

In `MarketThree`, both the `add_farm` and `add_market_emission` instructions push new entries to their respective vectors (`farms` and `emission`) without checking against `PERSONAL_TICK_YIELD_TRACKER_SIZE`.

This missing validation may result in vector overflows if farms or emissions exceed the safe maximum, and while `MarketEmissions::add_emission` enforces this limit, it is not utilized.

Remediation

Add validation to ensure `farms` and `emission` sizes are less than `PERSONAL_TICK_YIELD_TRACKER_SIZE`.

Patch

Resolved in [PR#3137](#).

Improper Size Allocations MEDIUM

OS-ELM-ADV-18

Description

1. The `ConfigurationOptions::SIZE_OF` constant incorrectly allocates 16 bytes for `max_lp_supply` and 2 bytes for `tick_space`, while their actual sizes are 8 and 4 bytes, respectively.

```
>_ src/state/market_three/market_three.rs RUST

impl ConfigurationOptions {
    pub const SIZE_OF: usize =
        // ln_fee_rate_root
        8 +
        // treasury_fee_bps
        2 +
        // min_lp_tick_amount
        8 +
        // epsilon_clamp
        8 +
        // max_lp_supply
        16 +
        // tick_space
        2 +
        [...]
}
```

2. `MarketThree::size_of` omits the `is_current_flash_swap` field from its size calculation, underestimating the required account space by 1 byte. Add `+ 1` for the missing boolean field.
3. The `split_epoch` field in `PrincipalShare` is a `u64` but is incorrectly counted as 4 bytes in `SIZE`. The size of `split_epoch: u64` should be 8 bytes instead of 4 bytes.

```
>_ src/state/position.rs RUST

impl PrincipalShare {
    pub const SIZE: usize =
        // tick_idx
        4 +
        // right_tick_idx
        4 +
        // split_epoch
        4 +
        // lp_share
        8 +
        PersonalYieldTracker::SIZE * PERSONAL_TICK_YIELD_TRACKER_SIZE;
    [...]
}
```

4. In the size calculation for `trackers: Vec<PersonalYieldTracker>` ensure to include the 4-byte vector length prefix required by Borsh serialization.
5. `PrincipalShareTrackers::size_of` incorrectly calls `PersonalYieldTrackers::static_size_of`, which calculates yield tracker sizes instead of principal share sizes. It should call `PrincipalShareTrackers::static_size_of` to correctly compute the storage size for `PrincipalShare`.

```
>_ src/state/position.rs
```

RUST

```
impl PrincipalShareTrackers {
    [...]
    pub fn size_of(&self) -> usize {
        PersonalYieldTrackers::static_size_of(self.trackers.len())
    }
}
```

6. In `withdraw_liquidity`, `deposit_liquidity`, and `market_accrue_emission` (shown below) instructions, `LpPosition::static_size_of` expects the number of farm emissions, but the code incorrectly utilizes the number of market emission trackers, resulting in incorrect account size calculations. Ensure the first parameter of `LpPosition::static_size_of` is `market.lp_farm.farm_emissions.len`.

```
>_ src/instructions/market_three/market_accrue_emission.rs
```

RUST

```
#[event_cpi]
#[derive(Accounts)]
#[instruction(emission_index: u16)]
pub struct MarketAccrueEmission<'info> {
    [...]
    #[account(
        mut,
        has_one = owner,
        has_one = market,
        realloc = LpPosition::static_size_of(
            market.emissions.trackers.len(),
            compute_ticks_amount_in_range_with_idx(
                &mut ticks.data.borrow_mut()[8..],
                lp_position.lower_tick_idx, lp_position.upper_tick_idx,
            ),
        ),
        realloc::payer = owner,
        realloc::zero = false,
    )]
    pub lp_position: Box<Account<'info, LpPosition>>,
    [...]
}
```

Remediation

Include the above size updates.

Patch

1. Issue #1 resolved in [PR#2955](#).
2. Issue #2 resolved in [PR#2955](#).
3. Issue #3 resolved in [PR#3006](#).
4. Issue #4 resolved in [PR#3006](#).
5. Issue #5 resolved in [PR#3006](#).
6. Issue #6 resolved in [PR#3006](#).

Balance Limit bypass due to Round Down Division

LOW

OS-ELM-ADV-19

Description

`LiquidityNetBalanceLimits::verify_limits` computes `percentage_increase` and `percentage_decrease` via floating-point division and flooring via integer casts, which silently rounds down the value. As a result, any true percentage change that is slightly above the configured threshold is incorrectly treated as acceptable. Thus, if the maximum allowed change is 10,000 (representing 100%), an attacker may exploit the truncation to push the actual change above the limit (such as increasing the balance by 100.009% during a deposit) without triggering the check. Consequently, this may be exploited to bypass the intended net balance restrictions.

```
>_ src/state/market_three/market_three.rs
```

RUST

```
pub fn verify_limits([...]) -> Result<()> {
    [...]
    if new_balance >= start_balance {
        // Handle positive change
        let balance_increase = new_balance - start_balance;
        let percentage_increase =
            (balance_increase as f64 / start_balance as f64 * 10000.0) as u32;
        require!(
            percentage_increase <= self.max_net_balance_change_positive_percentage,
            ExponentCoreError::NetBalanceChangeExceedsLimit
        );
    } else {
        // Handle negative change
        let balance_decrease = start_balance - new_balance;
        let percentage_decrease =
            (balance_decrease as f64 / start_balance as f64 * 10000.0) as u16;
        require!(
            percentage_decrease <= self.max_net_balance_change_negative_percentage,
            ExponentCoreError::NetBalanceChangeExceedsLimit
        );
    }
    Ok(())
}
```

Remediation

Ensure to round up the values of `percentage_increase` and `percentage_decrease`.

Patch

Resolved in [PR#3089](#).

Inflated Emission Index Initialization LOW

OS-ELM-ADV-20

Description

Currently, `lp_share_index` in `MarketEmissions::add_emission` is incorrectly initialized to `Number::ONE` instead of `Number::ZERO`. Consequently, new LP emissions start with a nonzero index, resulting in inflated rewards for LPs when they first claim. The index should start at zero so that rewards are accurately tracked from the moment the emission is added. Although currently this function is not utilized, any future utilization may result in issues.

```
>_ src/instructions/market_three/sell_yt.rs
```

RUST

```
impl MarketEmissions {  
    [...]  
    pub fn add_emission(&mut self, token_escrow: Pubkey) {  
        self.trackers.push(MarketEmission {  
            token_escrow,  
            lp_share_index: Number::ONE,  
            last_seen_staged: 0,  
        });  
  
        assert!(self.trackers.len() <= PERSONAL_TICK_YIELD_TRACKER_SIZE, "can't be more than max  
            ↳ size")  
    }  
}
```

Remediation

Set `lp_share_index` to `Number::ZERO`.

Patch

Resolved in [PR#3138](#).

Missing Final Fee Index Recording LOW

OS-ELM-ADV-21

Description

The swap logic updates the global fee-growth indices (`fee_growth_index_global_pt` and `fee_growth_index_global_sy`) as it processes each interval, but these final values are never written into the `ModifiedTicks` record. As a result, the swap event (`SwapOutcome`) emits incomplete fee-state information, preventing indexers and off-chain systems from reconstructing post-swap fee accounting.

Remediation

Update `ModifiedTicks` record with the values of `fee_global_after_pt` and `fee_global_after_sy`.

Patch

Resolved in [PR#3088](#).

Unrestricted Market Initialization LOW

OS-ELM-ADV-22

Description

`initialize_market` instruction in `market_three_init` currently lacks access control, allowing anyone to create a `MarketThree` instance. Since each market utilizes a `seed_id` (with only 256 available values) for account derivation, an attacker may repeatedly call this function to occupy all possible `seed_ids`, effectively blocking legitimate market creation.

```
>_ src/instructions/market_three/admin/market_three_init.rs
```

RUST

```
pub fn handler<'info>([...]) -> Result<MarketThreeInitEvent> {  
    // make the market account from a factory  
    msg!("Creating market struct: {}", 8 + std::mem::size_of::<Ticks>());  
  
    assert!(lower_tick_key < current_price_apy && upper_tick_key > current_price_apy);  
    let current_spot_price = (1.0 + (current_price_apy as f64) /  
        ↳ TICK_KEY_BASE_POINTS).ln().exp();  
    let market = make_market(  
        &ctx,  
        ln_fee_rate_root,  
        cpi_accounts,  
        cpi_core_accounts,  
        fee_treasury_bps,  
        min_lp_tick_amount,  
        epsilon_clamp,  
        max_lp_supply,  
        current_spot_price,  
        tick_space,  
        seed_id,  
    )?;
```

Remediation

Expand the `seed_id` capacity beyond 256.

Patch

Resolved in [PR#2955](#).

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-ELM-SUG-00	The module lacks the ability to update fee treasury and admin accounts, and the admin change should be made a two-step process to prevent irreversible ownership transfers.
OS-ELM-SUG-01	Recommendation for refactoring the codebase for better functionality and addressing inconsistencies, including the removal of unnecessary code.
OS-ELM-SUG-02	The current implementation allows anyone to create a <code>MarketThree</code> and supply malicious <code>cpi_sy_accounts</code> or <code>cpi_core_accounts</code> , risking adverse behavior.
OS-ELM-SUG-03	In <code>compute_token_needs</code> , the <code>sy_need</code> calculation incorrectly utilizes the unclamped price range instead of <code>delta_c_total</code> , creating inconsistency with the liquidity calculation.

Missing Functionality to Update Admin and Treasury Fields

OS-ELM-SUG-00

Description

`modify_market_setting` currently lacks support for updating the `token_fee_treasury_sy`, `token_fee_treasury_pt`, and `admin` fields, which limits flexibility. Extend the admin action enum to include support for updating these fields. Additionally, the admin change should follow a two-step process. The current admin should first propose the new admin, and the proposed admin must then explicitly accept the role. This prevents inadvertent and irreversible ownership changes.

Remediation

Implement the above suggestions.

Patch

Resolved in [PR#2956](#).

Code Refactoring

OS-ELM-SUG-01

Description

1. In `modify_farm`, without verifying that `new_expiration_timestamp >= current_timestamp`, the farm may be set to expire in the past, disrupting emission schedules and resulting in incorrect token budget calculations. Check `new_expiration_timestamp` is greater than or equal to `current_timestamp`.
2. `modify_market_setting` instruction incorrectly computes `additional_rent` utilizing `minimum_balance(new_size - old_size)`, which includes Solana's fixed account storage overhead (`ACCOUNT_STORAGE_OVERHEAD`) in the delta calculation. This approach overestimates the required lamports when reallocating. Instead, compute the difference between the total rents before and after resizing (`minimum_balance(new_size) - minimum_balance(old_size)`). This ensures only the actual incremental rent is transferred, avoiding unnecessary overpayment.

```
>_ src/instructions/market_three/admin/modify_market_setting.rs
```

RUST

```
#[access_control(ctx.accounts.validate())]
pub fn handler(ctx: Context<ModifyMarketSetting>, action: MarketAdminAction) -> Result<()>
{
    let market = &mut ctx.accounts.market;
    match action {
        [...]
        MarketAdminAction::ChangeCpiAccounts {
            cpi_accounts,
            cpi_core_accounts,
        } => {
            [...]
            if new_size > old_size {
                let additional_rent = Rent::get()?.minimum_balance(new_size - old_size);
                anchor_lang::system_program::transfer(
                    CpiContext::new(
                        ctx.accounts.system_program.to_account_info(),
                        anchor_lang::system_program::Transfer {
                            from: ctx.accounts.signer.to_account_info(),
                            to: market.to_account_info(),
                        },
                    ),
                    additional_rent,
                )?;
            }
        }
    }
    Ok(())
}
```

3. In `ticks::split_principals_after_insert`, calling `m.value.update_tick_emissions` for the new tick is redundant because `init_tick_emission_on_split` already initializes its emissions. Remove the call to `update_tick_emissions`.

```
>_ src/state/ticks.rs RUST

pub fn split_principals_after_insert(...) -> Result<()> {
    [...]
    {
        let m = self.ticks_tree.get_node_mut(new_idx);
        m.value.init_tick_emission_on_split(emission_position_indicies,
            ↪ market_emission_indicies);
        m.value.update_tick_emissions(market_emission_indicies);
        m.value.last_split_epoch = last_split_epoch;
        [...]
    }
    Ok(())
}
```

4. The current code in `position::accrue_fees_emissions_when_remove` truncates `diff` before multiplying by `share_to_remove`, losing fractional precision and underestimating emissions. Instead, multiply and then floor (`(diff * Number::from(share_to_remove)).floor_u64()`) to preserve accuracy. Additionally, the returned `staged_amounts` array is never utilized and may be removed.

```
>_ src/state/position.rs RUST

pub fn accrue_fees_emissions_when_remove(&mut self, tick: &Tick, share_to_remove: u64) ->
    ↪ Result<[u64; PERSONAL_TICK_YIELD_TRACKER_SIZE]> {
    let mut staged_amounts: [u64; PERSONAL_TICK_YIELD_TRACKER_SIZE] =
        ↪ [0u64; PERSONAL_TICK_YIELD_TRACKER_SIZE];
    for index in 0..PERSONAL_TICK_YIELD_TRACKER_SIZE {
        let diff = tick.emissions.trackers[index].last_position_index -
            ↪ self.emissions.trackers[index].last_seen_index;
        staged_amounts[index] = diff.floor_u64() * share_to_remove;
        [...]
    }
    Ok(staged_amounts)
}
```

5. In `deposit_liquidity`, if `lower_tick_key` equals `upper_tick_key`, the position spans zero tick intervals, rendering the liquidity deposit logically invalid. Enforce `lower_tick_key != upper_tick_key`.

6. `ticks::get_amount_ticks_in_range` is purely read-only, so utilizing `iter_mut` unnecessarily requests mutable access. Replace it with `iter` to improve safety and clarity.

```
>_ src/state/ticks.rs RUST

pub fn get_amount_ticks_in_range(&mut self, lower_key: u32, upper_key: u32) ->
    Result<usize> {
    [...]
    let mut counter_index = if left_idx_optional != SENTINEL {
        left_idx_optional
    } else {
        counter += 1;
        let start_key = if let Some(start_key_from_tree) =
            self.ticks_tree.iter_mut().find(|tick| *tick.0 > lower_key)
        {
            *start_key_from_tree.0
        } else {
            SENTINEL
        };
        self.ticks_tree.get_addr(&start_key)
    };
    [...]
    counter += 1; // no upper tick exist
    return Ok(counter);
}
```

7. In `buy_yt`, `sell_yt`, and `wrapper_provide_liquidity`, it will be appropriate to explicitly include the `exponent_core` program, as these instructions rely on core CPI calls.

Remediation

Incorporate the above refactors.

Patch

1. Issue #1 resolved in [PR#3008](#).
2. Issue #2 resolved in [PR#3007](#).
3. Issue #3 resolved in [PR#3091](#).
4. Issue #4 resolved in [PR#3092](#).
5. Issue #5 resolved in [PR#3143](#).
6. Issue #6 resolved in [PR#3093](#).
7. Issue #7 resolved in [PR#3142](#).

Validation for CPI Accounts

OS-ELM-SUG-02

Description

Since anyone may create a `MarketThree`, a malicious user may set `cpi_sy_accounts` or `cpi_core_accounts` to untrusted programs. Add strict validations for `CpiAccounts` and `CpiCoreAccounts` as listed below to ensure these CPI accounts reference only the correct accounts:

1. `CpiAccounts.get_sy_state`:
 - (a) must match `vault.cpi_accounts.get_sy_state`.
2. `CpiAccounts.deposit_sy`:
 - (a) `position` must be `lp_position`.
 - (b) `token_sy_depositor` must be `market.token_sy_escrow`.
3. `CpiAccounts.withdraw_sy`:
 - (a) `position` must be `lp_position`.
 - (b) `token_sy_dst` must be `market.token_sy_escrow`.
4. `CpiAccounts.claim_emission`:
 - (a) `position` must be `lp_position`.
5. `CpiAccounts.get_position_state`:
 - (a) `position` must be `lp_position`.
6. `CpiCoreAccounts.strip_sy`:
 - (a) `vault` must be `market.vault`.
 - (b) `sy_src` must be `market.token_sy_escrow`.
 - (c) `pt_dst` and `yt_dst` must be `market.token_pt_escrow` and `market.token_yt_escrow` respectively.
7. `CpiCoreAccounts.unstrip_sy`:
 - (a) `vault` must be `market.vault`.
 - (b) `sy_dst` must be `market.token_sy_escrow`.
 - (c) `pt_src` and `yt_src` must be `market.token_pt_escrow` and `market.token_yt_escrow` respectively.

Remediation

Add the above stated account validation checks. Additionally, it will be appropriate to store `ctx.accounts.lp_position` as a field in `MarketThree`, and tracking the position account index for each (`sy_program`, `cpi_ix`) pair, since the index of the position account may vary across different `sy_programs`.

Discrepancy in SY Need Calculation

OS-ELM-SUG-03

Description

In `math::compute_token_needs`, within the in the below-range case, the function clamps `(price_eff_lower - price_eff_upper)` into `delta_c_total` to prevent zero or tiny widths, and utilizes this clamped value when computing liquidity. However, subsequently `sy_need` is mistakenly calculated utilizing the unclamped width, resulting in the `SY` requirement to become inconsistent with the liquidity target.

```
>_ src/utils/math.rs
```

RUST

```
pub fn compute_token_needs([...]) -> (u64, u64) {  
    // Decide region and compute target  $\Delta L$  and token needs under A-anchored formulas  
    if spot_price_current <= lower_price {  
        // Below range: SY only  
        let delta_c_total = (price_eff_lower - price_eff_upper).max(epsilon_clamp);  
        let liquidity_from_sy = (max_sy as f64) / delta_c_total;  
        let liquidity_target = liquidity_from_sy;  
        let sy_need = (liquidity_target ) * (price_eff_lower - price_eff_upper);  
  
        return (sy_need.floor() as u64, 0);  
    }  
    [...]  
}
```

Remediation

Utilize `delta_c_total` when calculating `sy_need` in the below-range case.

Patch

Resolved in [PR#3139](#).

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the [General Findings](#).

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
 - Improperly designed economic incentives leading to loss of funds.
-

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
 - Exploitation involving high capital requirement with respect to payout.
-

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
 - Forced exceptions in the normal user flow.
-

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.
-

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
 - Improved input validation.
-

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.